UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CURSO DE TECNOLOGIA EM SISTEMAS PARA INTERNET
CÂMPUS GUARAPUAVA

LUIZ FELIPE BASILE RIBEIRO FILHO

RU DIGITAL: UM SISTEMA PARA GERENCIAMENTO DE SALDO FINANCEIRO DE CLIENTES DE RESTAURANTES UNIVERSITÁRIOS

PROJETO DE TRABALHO DE CONCLUSÃO DE CURSO

GUARAPUAVA
2018

LUIZ FELIPE BASILE RIBEIRO FILHO

RU DIGITAL: UM SISTEMA PARA GERENCIAMENTO DE SALDO FINANCEIRO DE CLIENTES DE RESTAURANTES UNIVERSITÁRIOS

Projeto de Trabalho de Conclusão de Curso apresentado ao Curso de Tecnologia em Sistemas para Internet da Universidade Tecnológica Federal do Paraná, como requisito parcial para a obtenção do título Tecnólogo.

Orientador (a): Prof. Dr. Roni Fabio Banaszewski

Universidade Tecnológica Federal do Paraná

GUARAPUAVA
2018

RESUMO

FILHO, Luiz Felipe Basile Ribeiro. RU Digital: Um Sistema para Gerenciamento de Saldo Financeiro de Clientes de Restaurantes Universitários. 2018. 20 f. Trabalho de Conclusão de Curso – Curso de Tecnologia em Sistemas para Internet, Universidade Tecnológica Federal do Paraná. Guarapuava, 2018.

Um problema muito comum presente na maioria dos restaurantes universitários de hoje em dia é a falta de um sistema que permita a inserção de crédito e posterior controle dos gastos pelo cliente, onde o mesmo possa gerenciar o seu saldo e tenha a possibilidade de verificar seu extrato de conta. A maioria dos restaurantes universitários não possui um sistema que possibilite um saldo virtual, ou seja, uma espécie de crédito pré-pago aos seus clientes da instituição. Neste sentido, este projeto consiste no desenvolvimento de um sistema para que a gerência dos restaurantes universitários possam ter controle dos saldos de seus clientes e estes clientes tenham mais flexibilidade de formas de pagamento e também no acompanhamento do gasto de seu saldo. O sistema proposto é denominado RU-Digital. O sistema têm como objetivo atender as demandas de restaurantes universitários para o gerenciamento de contas dos clientes e movimentações financeiras de crédito por diversas formas de pagamento e movimentações de débito relacionadas às despesas dos clientes.

Palavras Chaves: Restaurante universitário. Digital. Sistema. Saldo Virtual. Online.

ABSTRACT

FILHO, Luiz Felipe Basile Ribeiro. RU Digital: Um Sistema para Gerenciamento de Saldo Financeiro de Clientes de Restaurantes Universitários. 2018. 20 f. Trabalho de Conclusão de Curso – Curso de Tecnologia em Sistemas para Internet, Universidade Tecnológica Federal do Paraná. Guarapuava, 2018.

A very common problem present in most university restaurants today is the lack of a system that allows the insertion of credit and subsequent control of the expenses by the client, where he can manage his balance and have the possibility to verify his statement account. Most university restaurants do not have a system that allows a virtual balance, that is, a kind of prepaid credit to their clients of the institution. In this sense, this proposal consists in the development of a system so that the management of university restaurants can control the balances of their clients and these clients have more flexibility of forms of payment and also in the monitoring of the expenditure of their balance. The proposed system is called RU-Digital. The program aims to meet the demands of university restaurants for the management of customer accounts and financial movements of credit for various forms of payment and debit transactions related to customer expenses.
Keywords: University restaurant. Digital. System. Virtual Balance. Online.

LISTA DE FIGURAS

Figura 1: No-Excel: Um sistema para gerenciamento e controle de saldo
Figura 2: Rucard, sistema para compra de créditos para o restaurante da USP
Figura 3: SRU, sistema de reserva de tickets do restaurante da UNESP
Figura 4: Ciclo do Scrum
Figura 5: Projeto da Arquitetura do Sistema
Figura 6: Ru-Digital: Tela inicial do Sistema
Figura 7: RU Digital: Tela de Cadastro de Clientes
Figura 8: RU DIgital: Tela de Conta do Usuário
Figura 9: RU Digital: Tela de Edição de Conta do Cliente
Figura 10: RU Digital Tela de Extrato de Consumo do Cliente
Figura 11: RU Digital Tela de Login dos Usuários
Figura 12: RU Digital Tela de Extrato de Cadastro do Gerente
Figura 13: RU Digital Tela de Extrato de Inserção de Créditos para Clientes
Figura 14: RU Digital Tela de Lançamento de Consumos
Figura 15: RU Digital Tela de Cadastro de Produtos e Preços
Figura 16: RU Digital Modelo do Banco de dados

LISTA DE QUADROS

Tabela 1 Histórias do Cliente

Tabela 2 Histórias do Administrador e do Gerente

Tabela 3 História dos Visitantes

LISTA DE ABREVIATURAS E SIGLAS

CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
Java EE	Java Enterprise Edition
JSP	Java Server Pages
MVC	Model View Controller
ORM	Object Relational Mapping
REST	Representational State Transfer
SPA	Single Page Application
SRU	Sistema do Restaurante Universitário
SSL	Secure Socket Layer
STS	Spring Tools Suite
UTFPR	Universidade Tecnológica Federal do Paraná
XML	Extensible Markup Language

SUMÁRIO

1 INTRODUÇÃO **9**

2 OBJETIVOS **10**

2.1 Objetivo Geral 10

2.2 Objetivos Específicos 10

3 REVISÃO DA LITERATURA **11**

3.1 Contextualização 12

3.2.1 Rucard 14

3.2.2 SRU 14

4 FUNDAMENTAÇÃO TEÓRICA **16**

4.1 Tecnologias do Lado Cliente 16

4.2 Tecnologias do Lado Servidor 17

5 PROCEDIMENTOS METODOLÓGICOS **23**

5 Análise e Projeto do Sistema 24

5.1 Análise do Sistema 24

5.2 PROJETO DO SISTEMA **27**

6 CONSIDERAÇÕES FINAIS **42**

7 REFERÊNCIAS **42**

1 INTRODUÇÃO

O restaurante universitário da UTFPR tem como objetivo atender com qualidade, servindo as refeições de forma balanceada e com cuidados de higiene, mantendo um custo acessível a toda a comunidade acadêmica. Porém, as filas que são consequência de um grande movimento durante as refeições influenciam de forma negativa na qualidade prestada em alguns desses restaurantes.

Conforme José Airton Azevedo dos Santos (2012), grande parte dos restaurantes universitários utilizam o modelo de refeição self-service (em português auto-serviço), que dispensam o atendimento de garçons. Apesar desse tipo de restaurante ser predominante nos centros universitários, a maioria desses apresentam filas, já que o movimento de clientes ocorre em um curto espaço de tempo. Isto ocorre, principalmente, pelo fato de que as universidades possuem um horário comum destinado a refeições para todos os cursos de mesmo período.

Um fator predominante para ocasionar as filas nesses ambientes é ocasionada pela demora no atendimento. A maioria dos restaurantes universitários ainda não possuem um sistema para agilizar o atendimento de seus clientes. Alguns restaurantes tentam oferecer um serviço de saldo pré-pago para agilizar o pagamento e a facilidade de troco, mas as informações são guardadas em meios ineficientes e tendentes a erros, tal como em planilhas eletrônicas ou controle em fichas de papel. No que se refere ao ato de dispor o troco, este pode ser visto como um dos principais fatores que contribuem para com o aumento das filas.

Muitas vezes o atendente do estabelecimento tem dificuldades em retornar o troco, uma vez que nem sempre possui moedas no caixa. Além disso, o cliente pode se sentir desconfortável carregando uma quantidade exagerada de moedas em seus bolsos, quando esta quantia poderia ser empregada em seu saldo para usar em refeições futuras, caso o controle desses valores fosse realizado por um meio mais confiável.

A ausência de um sistema que atenda a demanda de restaurante universitários

para evitar o troco e automatizar todo o processo financeiro justifica a construção de um sistema capaz de efetuar essas tarefas. Este sistema permitiria também o acompanhamento pelo cliente de todas suas transações no restaurante de forma concentrada e facilitada.

Como motivação para desenvolver o sistema, foi realizada pelo autor desta proposta, uma pesquisa via redes sociais com estudantes da UTFPR do câmpus Guarapuava sobre o interesse de um sistema para gerenciamento de saldo virtual. Em concordância com as situações supracitadas, foi observado que 91,5% dos estudantes acreditam que é interessante um sistema que possibilite a inserção de crédito no restaurante universitário. O formulário e suas respostas é apresentado no Apêndice A. Com isso, essa pesquisa revela a importância e boa receptividade para implantação desse sistema.

2 OBJETIVOS

2.1 Objetivo Geral

O objetivo deste projeto é o desenvolvimento de um sistema para gerenciar saldos financeiros de clientes com transações de crédito via múltiplos meios de pagamento e transações de débito relacionadas a despesas com refeições realizadas em restaurantes universitários, principalmente da UTFPR.

2.2 Objetivos Específicos

- Permitir ao administrador o acompanhamento das entradas no restaurante através do acompanhamento atual do saldo em caixa, saldo creditado e saldo debitado;
- Permitir ao cliente inserir crédito em sua conta virtual e acompanhar suas transações de crédito e débito (em forma de despesas) em um extrato datado e detalhado;
- Facilitar o acompanhamento do gerente do restaurante às transações realizadas (inserção de crédito ou consumo), evitando possíveis divergências e contestações

de clientes;

- Oferecer a possibilidade ao dono do restaurante de uso de saldo particular (i.e. consumo somente no restaurante onde inseriu crédito) ou global (i.e. o saldo pode ser consumido em vários restaurantes) em seu restaurante, dependendo do seu interesse e necessidade;
- Proporcionar informações aos clientes sobre horário de funcionamento, cardápio do dia, preços e localização;
- Oferecer segurança sobre as transações financeiras.

3 REVISÃO DA LITERATURA

Neste capítulo, a seção 3.1 apresenta uma contextualização sobre a origem do RU Digital e trabalhos já realizados neste sentido. A seção 3.2 apresenta os sistemas similares já existentes no mercado. Por fim, a seção 3.3 apresenta as considerações finais sobre o capítulo.

3.1 Contextualização

Em 2017, o autor deste projeto de conclusão de curso participou de um projeto de extensão para o desenvolvimento de um sistema para automatizar as transações financeiras no restaurante da UTFPR do câmpus Guarapuava (RIBEIRO FILHO, 2017). Este sistema foi denominado No-Excel.

Ao perceber como ocorria a manipulação de créditos dos clientes no restaurante da UTFPR do câmpus Guarapuava, surgiu a ideia de criar um sistema online para aumentar principalmente a segurança nessas transações. O sistema criado conta com o cadastro dos clientes e telas próprias para a realização do crédito e débito de valores. Os clientes podem também acompanhar o consumo de seus saldos via extrato após a autenticação no sistema. Na tela principal, o usuário tem acesso a conta, ao extrato e ao perfil, tal como ilustrado na Figura 1.

Figura 1: No-Excel: Um sistema para gerenciamento e controle de saldo pré-carregado para restaurantes universitários

Fonte: Ribeiro FIlho (2017)

Apesar das funcionalidades implementadas, o sistema No-Excel não foi implantado principalmente por apresentar algumas limitações. Uma limitação principal é a

falta de suporte à múltiplos restaurantes. Ademais, o sistema não conta com uma área para sugestões e também não possui uma página para apresentar o cardápio.

Na verdade, o No-Excel consiste em um sistema mais específico que o RU Digital, pois o objetivo principal do No-Excel é atender o restaurante da UTFPR-Guarapuava enquanto o RU Digital visa funcionar em qualquer restaurante universitário ou até mesmo em uma lanchonete universitária. Outro diferencial do RU Digital é a proposta do pagamento online, permitindo uma maior autonomia do cliente, essa funcionalidade não foi implementada no No-Excel.

Algumas análises foram realizadas para considerar a agregação das novas funcionalidades ao No-Excel. Porém, considerando a experiência do autor adquirida com a implementação deste sistema, a dificuldade de manutenção deste sistema e desejo de aprender uma nova tecnologia, foi decidido pela implementação de um novo sistema.
Estado da arte

Os restaurantes universitários geralmente são integrados com sistemas da instituição em que atuam para um maior controle do consumo e valor monetário arrecadado. Estes sistemas são usados para evitar fraudes, uma vez que para certos perfis de clientes, a universidade ou o governo contribuem com o pagamento de parte do preço das refeições.

Como os restaurantes universitários atuam com a intenção de oferecer um bom serviço com baixo custo, muitos não contam com diferentes formas de pagamento, tal como cartão de entidades financeiras, uma vez que isto acarreta no pagamento de uma taxa aos bancos por transação ou mensalidade. Por exemplo, a Moderninha Pro do PagSeguro (máquina de cartões) cobra 2,39% por transação de débito (PAGSEGURO, 2018) e as máquinas banda larga da empresa Cielo possuem uma taxa de R$ 144,90 por mês (CIELO, 2017). Desta forma, uma alternativa que agilizaria o atendimento e ofereceria um conforto maior aos clientes seria a implantação de um sistema que gerenciasse de forma confiável um saldo virtual (com depósitos em dinheiro em espécie) e suas transações para o restaurante.

São poucas alternativas de sistemas com este foco presentes no mercado, pois as instituições educacionais visam baratear os custos e por isso geralmente o pagamento ocorre em dinheiro em espécie. A seguir são apresentadas algumas iniciativas que tentaram implantar estes mecanismos em instituições educacionais.

3.2.1 Rucard

O Rucard é o sistema utilizado no restaurante da USP, o qual possui a finalidade de compras de crédito via boleto. Ele funciona através de um cartão para consumo apenas no restaurante da USP que pode ser carregado pelo site por meio do pagamento de boleto. O valor de crédito é escolhido pelo cliente no momento da solicitação. O histórico de compras de créditos fica disponível ao cliente que pode gerar o boleto quando quiser, conforme pode ser observado na Figura 2.

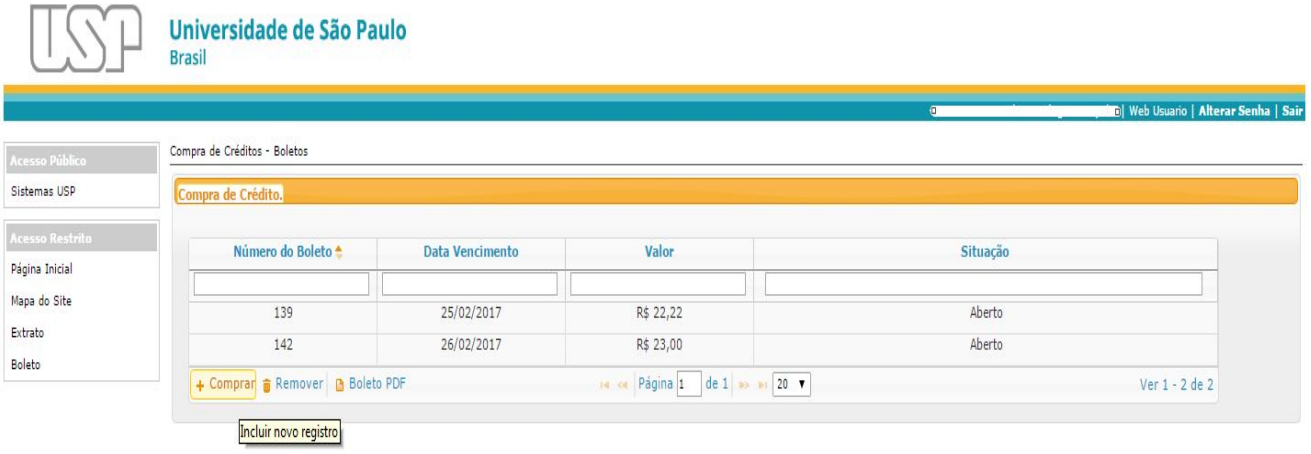

Figura 2: Rucard, sistema para compra de créditos para o restaurante da USP

Fonte: Rucard(2017)

O Rucard tem a opção de extrato, disponibiliza informações de data, quantidade, tipo e valor de movimentações que já aconteceram. O sistema possui apenas uma forma de pagamento (boleto). Não possui o cardápio do dia e nem área de sugestão.

3.2.2 SRU

O SRU é um sistema para reserva e transferência de tickets que podem ser utilizados no restaurante universitário da UNESP. O cliente primeiramente deve efetuar a reserva do ticket via sistema e após garantir sua reserva, o mesmo deve comparecer ao

restaurante no horário de funcionamento para adquiri-lo. Para o cliente consumir, o mesmo deve possuir um cartão, ou seja, caso a pessoa não tenha comprado o ticket a catraca não é liberada.

O SRU é um sistema web. O sistema em si é gratuito, porém, é cobrado uma taxa para adquirir o cartão. Cada ticket é para um dia específico, ou seja, caso o cliente compre o ticket e não use ou transfira para alguém, o restaurante não fará o reembolso. O SRU possui o cardápio semanal disponibilizado no site para consulta como pode ser visto na Figura 3.

Cardápio

Período	Prato Principal	Opção Vegetariana	Guarnição	Acompanhamento	Salada	Sobremesa	Suco
Almoço - 09/04/2018	Frango grelhado	Omelete	Farofa de banana	Arroz, feijão	Beterraba ralada	Doce de leite	Caju
Almoço - 10/04/2018	Quibe frito	Brócolis gratinado	Chuchu	Arroz, feijão	Tomate + cebola	Abacaxi	Goiaba
Almoço - 11/04/2018	Linguiça assada	PTS	Abobrinha	Arroz, feijão	Alface + cenoura	Melancia	Maracujá
Almoço - 12/04/2018	Nhoque	Nhoque vegetariano	----------	Arroz, feijão	Pepino	Gelatina	Laranja
Almoço - 13/04/2018	Estrogonofe de carne	Estrogonofe vegetariano	Batata palha	Arroz, feijão	Repolho bicolor	Banana	Uva

Anterior 1 2 Seguinte

Figura 3: SRU, sistema de reserva de tickets do restaurante da UNESP

Fonte: SRU (2017)

O SRU tem um custo inicial. Além disso, o SRU não permite o pagamento do ticket online, para isso o cliente precisa se deslocar até o local. Os tickets possuem uma validade, pois cada ticket adquirido deve ser consumido em uma refeição específica. Já no RU Digital, não existe custo algum, apenas com o RA ou identidade em mãos o cliente pode utilizar o sistema. Além disso, o crédito inserido no RU Digital não terá validade e nunca será expirado, evitando o descontentamento dos clientes. Por fim, o SRU é um sistema focado em reservas, enquanto o RU Digital é mais focado no gerenciamento das refeições nos restaurantes.

4 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo, é abordado de forma sucinta as tecnologias que serão usadas ao longo do desenvolvimento desse projeto. Em uma arquitetura cliente-servidor, uma sistema é desmembrado em dois ambientes distintos de execução, ou seja, uma parte do sistema executa no lado cliente (i.e. navegadores web) e outra parte no lado servidor (i.e. servidores web). Geralmente, os termos adotados para se referir a estes dois ambientes de execução derivam dos termos em inglês Front-end e Back-end ou também dos termos client-side e server-side. Mais precisamente, na seção 4.1 são descritas as tecnologias a serem usadas no lado cliente e na seção 4.2 são descritas as tecnologias a serem adotadas para o lado servidor da aplicação. Na seção 4.3 é descrito sobre a arquitetura utilizada no sistema. As ferramentas do sistema são descritas na seção 4.4. A questão da segurança é tratada no capítulo 4.5. e por fim na seção 4.6 são apresentadas as considerações finais sobre o capítulo.

4.1 Tecnologias do Lado Cliente

Conforme Jon Duckett(2015), o HTML é uma linguagem utilizada para a criação da estrutura de páginas web. Seu acrônimo vêm do inglês Hypertext Markup Language, que em português significa Linguagem de Marcação de Hipertexto. O HTML foi criado por Tim Berners Lee(EIS, 2011) na década de 90. O HTML é formado por tags que são marcadores utilizados para mostrar qual informação uma página deve exibir. As tags são definidas por palavras entre parênteses angulares (< e >) e podem conter atributos para modificar seus comportamentos.

Por sua vez, o termo CSS é originado do inglês Cascading Style Sheets, que em português significa folhas de estilos em cascata. O CSS é usado para definir a aparência e estilo de um documento em HTML (SCHIMITT, 2010).

Para aumentar a produtividade na criação de aplicações com aparência bonita e uso amigável, existem alguns frameworks CSS que disponibilizam um conjunto rico de estilos pré-definidos e prontos para uso. Como exemplo está o framework MaterializeCss (MATERIALIZECSS, 2014), que implementa o modelo de estilo adotado e desenvolvido recentemente pelo Google, mais precisamente denominado de Material Design (MOURA, 2017). Além da disponibilização de estilo de diferentes componentes gráficos, o MaterializeCss fornece recursos para criar páginas responsivas, que permite às páginas do sistema se adaptarem aos diferentes tamanhos de telas de dispositivos existente, algo indispensável atualmente.

Por fim, conforme aponta Duckett(2016) o JavaScript é uma linguagem de programação leve, interpretada, baseada em objetos com funções de primeira classe desenvolvida por Brendan Eich. Atualmente, o JavaScript é a principal linguagem de navegadores web e também pode ser utilizada fora do ambiente de um navegador web, podendo inclusive ser usada para implementar aplicações no lado do servidor por meio da plataforma Node.js (NODEJS, 2018) .

4.2 Tecnologias do Lado Servidor

O lado servidor será desenvolvido utilizando a linguagem Java com o ecossistema Spring, o qual é formado por vários frameworks com diferentes propósitos, tais como para a criação da arquitetura Modelo-Visão-Controle (MVC) do sistema, publicação de serviços web conforme a arquitetura REST e acesso a registros em bancos de dados relacionais via práticas de Mapeamento Objeto-Relacional (ORM). Neste sentido, o lado servidor será implementado seguindo o padrão arquitetural MVC, sendo que as funcionalidades também serão disponibilizadas em forma de serviços, implementados conforme o estilo arquitetural REST, sendo os dados armazenados no sistema de banco de dados relacional MySQL. Essas tecnologias serão descritas nas seções a seguir.

4.3.1 Arquitetura REST e MVC

O ecossistema Spring permite com grande facilidade adotar de uma única vez as arquiteturas REST e MVC, agregando ao sistema as vantagens de ambos padrões.

A sigla REST é originada do inglês Representational State Transfer, traduzido para português como Transferência de Estado Representacional. O REST é um modelo de arquitetura que define diversas restrições e propriedades baseados no protocolo HTTP. O REST define como os métodos básicos disponíveis no protocolo HTTP devem operar. Como exemplos destes métodos do HTTP estão: GET, POST, PUT e DELETE.

Por sua vez, o MVC tem por objetivo separar a aplicação em 3 camadas, das quais derivam o seu nome. Mais precisamente, a camada de modelo é responsável pela validação, leitura e escrita dos dados; sendo a camada de visão responsável pela exibição das informações ao usuário e por fim, a camada de controle sendo responsável por controlar as requisições dos usuários.

Com o uso integrado destas duas arquiteturas, as funcionalidade podem ser publicadas como serviços para acesso por outros sistemas ou principalmente por qualquer forma de implementação da camada de visão do padrão MVC. Com isso, versões futuras do lado cliente poderão ser implementadas como um sistema Single Page Application (SPA) em JavaScript (i.e Angular, React, Vue.js, etc) ou mesmo como um aplicativo nativo para plataformas de dispositivos móveis (e.g. Android, IPhone), não precisando mudar as camadas de Modelo-Controle do MVC. Além disso, graças a esta integração, a própria implementação do MVC pelo ecossistema Spring oferece a camada de visão integrada (i.e. JSP ou Thymeleaf) a fim de que a aplicação se comporte nativamente como um site web.

4.3.2 Spring

O ecossistema Spring consiste em um conjunto de frameworks que podem trabalhar juntamente para auxiliar na criação de aplicações Java para Web. O Spring foi criado como uma alternativa ao Java EE prezando pela simplicidade (AFONSO, 2017). A seguir são descritos alguns dos frameworks que compõe o Spring:

- Spring Boot: tem como objetivo facilitar as configurações iniciais para novos projetos que usam o Spring e reduzir ao máximo a necessidade de arquivos XML para configurações do projeto.
- Spring MVC: tem como objetivo permitir a construção de aplicações web flexíveis e robustas atendendo os padrões arquiteturais do MVC. Além disso atualmente o Spring MVC suporta o desenvolvimento na arquitetura REST.
- Spring Data: tem como objetivo disponibilizar de forma facilitada a manipulação de banco de dados relacionais em Java. Este framework facilita o uso do Java Persistence API (JPA), uma especificação Java para Mapeamento Objeto-Relacional (ORM). Por fim, o framework oferece algumas funcionalidades prontas, como ordenação das consultas e paginação dos registros.
- Spring Security: tem como objetivo cuidar da segurança da aplicação, além de tornar o desenvolvimento da autenticação e autorização mais fácil.

4.3.3 Thymeleaf

O Thymeleaf é uma biblioteca baseada em Java que é executado no lado do servidor e foi desenvolvida para facilitar a criação da camada de visão do modelo MVC com o Spring. O Thymeleaf disponibiliza recursos para gerar dinamicamente a parte do front-end da aplicação (HTML, CSS e JavaScript).

Para desenvolvimento das páginas poderia ser utilizado a tecnologia JSP que também possui a função de auxiliar na criação da camada de visão. Porém o Thymeleaf

possui algumas vantagens, estas são:

- Desenvolvimento de código HTML puro: para desenvolver é utilizado apenas HTML, sem a necessidade de códigos em outra linguagem na camada da visão;
- Link Expression: expressões que auxiliam no momento da programação evitando códigos repetitivos;
- Isolamento do front-end: ao usar o Thymeleaf é possível desenvolver as páginas sem a necessidade de um servidor para gerar o HTML.

4.4 Ferramentas do Sistema

O Spring Tools Suite (STS) é um ambiente para desenvolvimento baseado no Eclipse (Interface de Desenvolvimento Java desenvolvida pela IBM) e foi customizado pelo Spring para desenvolver aplicações usando essa tecnologia. O STS possui integração com outras tecnologias que facilitam no desenvolvimento, como por exemplo, servidores Pivotal, Maven e Git (usado para versionamento do código).

O STS conta com uma funcionalidade de ajuda para preenchimento de código, ou seja, ao começar a digitar ele apresenta opções para o desenvolvedor escolher e não precisar digitar todo o comando. Essa funcionalidade se apresenta tanto para arquivos XML quanto para arquivos Java (STS, 2018). O Spring Tools Suite será usado para desenvolvimento do projeto.

Como o STS dá suporte ao Maven, esta ferramenta será usada para a construção do projeto e gerenciamento das dependências de bibliotecas de código. Basicamente, o Maven é uma ferramenta para gerenciamento, construção e implantação de projetos, a qual foi criada para simplificar os processos para construção de um projeto em Java e aumentar a produtividade das equipes.

Outra ferramenta importante para o desenvolvimento de sistemas, refere-se ao Sistema Gerenciador de Banco de Dados (SGBD). Neste projeto, o SGBD a ser usado

será o MySQL. O MySQL foi criado na Suécia por David Axmark, Allan Larsson e Michael "Monty" Widenius. O Mysql é um dos SGBDs mais populares atualmente possui mais de 10 milhões de instalações no mundo. Conforme Pedro Pisa(2012), “O MySQL é um SGBD que possui código aberto. Ele é utilizado na maioria das aplicações gratuitas para gerenciar os dados. O MySQL utiliza a linguagem SQL que é a mais popular para inserir, acessar e gerenciar o banco de dados”.

Por fim, outra ferramenta indispensável para o desenvolvimento e execução de um projeto web, é o servidor web. Neste projeto, o servidor a ser utilizado será o Tomcat, um servidor web para aplicações Java. O Tomcat foi desenvolvido pela Apache Software Fundation e é software livre (TOMCAT, 2018). É um servidor leve e já vem integrado com o ecossistema Spring, facilitando e incentivando o seu uso no corrente projeto.

4.5 Segurança da Aplicação

O sistema que será desenvolvido precisa ser seguro e confiável, uma vez que trata de transações financeiras. Para tanto, será utilizado o algoritmo BCrypt para salvar senhas e dados sensíveis de forma criptografada no banco. Ademais, o sistema será implementado com o framework Spring Security que trata de diversos tipos de ataques. Como a segurança é um assunto importante e norteia o desenvolvimento deste projeto, alguns conceitos referentes a este assunto são melhor descritos nas subseções seguintes.

4.5.1 HTTPS/SSL

O SSL (Secure Socket Layer) é um protocolo de segurança e garante a veracidade, integridade e proteção na comunicação do cliente com o servidor. Essa segurança é garantida através da autenticação entre as partes envolvidas no uso do SSL. Nesse projeto será gerado um SSL para garantir a segurança dos dados e será

implantado no servidor com auxílio do Spring Boot. Geralmente, os serviços das autoridades certificadoras para uso do protocolo SSL são pagas, o que inviabiliza a aplicação desta tecnologia nos estágios iniciais deste projeto. Porém, uma alternativa gratuita é o LetsEncrypt (LETSNCRYPT, 2018). Ela permite certificar sites web gratuitamente sem grandes limitações a fim de garantir o nível de segurança desejado.

4.5.2 Bcrypt

O BCrypt é um algoritmo para criptografia de hashes e é baseado no algoritmo Blowfish (BCRYPT-OFFICIAL, 2013). O BCrypt foi desenvolvido por Niels Provos e Davi Maziéres em 1999. O BCrypt possui uma vantagem com relação aos outros algoritmos pois contém uma variável que é proporcional à quantidade de processamento necessário para criptografar a informação desejada, tornando-o resistente a ataques do tipo “força-bruta” (BCRYPT, 2018).

4.5.3 OWASP

É uma comunidade que disponibiliza gratuitamente informações de segurança para aplicações web. O site mantém uma lista atualizada com os 10 itens mais relevantes para aplicações(OWASP, 2017). No momento os itens definidos são: inserção de código, quebra de autenticação, exposição de dados sensíveis, entidades externas de XML, quebra de controle de acesso, configuração incorreta de segurança, cross-site scripting (XSS), desserialização insegura, utilização de componentes com vulnerabilidades conhecidas, log e monitoramento ineficientes.

As falhas de quebra de autenticação, entidades externas de XML, configuração incorreta de segurança, exposição de dados sensíveis e XSS são resolvidas com a utilização do Spring. As demais falhas listadas no TOP 10 serão tratadas com validações customizadas quando aplicáveis.

4.6 Conclusão

O sistema proposto será desenvolvido usando as tecnologias de frameworks mais

recentes a fim de evitar vulnerabilidades e aumentar a proteção dos dados contra ataques de segurança na web. Esta mesma preocupação se dará na instalação das ferramentas para implantação do sistema, incluindo políticas de segurança no servidor web e também no sistema operacional a ser utilizado para executar tal servidor. Enfim, todos os cuidados serão tomados seguindo as boas práticas de segurança para que além de ser funcional e resolver os problemas supramencionados dos restaurantes universitários, o sistema possa ser confiável para uso pelos proprietários de tais restaurantes e toda a comunidade acadêmica.

5 PROCEDIMENTOS METODOLÓGICOS

Nessa seção é descrito os procedimentos metodológicos que deverão ser adotados para o desenvolvimento do sistema. Como processo de desenvolvimento de software, será utilizado a metodologia Ágil Scrum.

O Scrum é uma metodologia ágil de desenvolvimento de software onde os projetos são divididos em ciclos que são chamados de Sprints. Estes ciclos ocorrem geralmente em períodos de duas a quatro semanas e representam a principal característica da iteração das metodologias ágeis.

As funcionalidades a serem implementadas no sistema serão descritas como histórias do usuário, sendo que um conjunto de histórias que define o escopo inicial do sistema é denominado de Product Backlog.As funcionalidade que serão implementadas em uma determinadas Sprint são escolhidas da lista de Product Backlog e são nomeadas como Sprint Backlog.

Os passos metodológicos são:

- Levantamento dos requisitos do sistema para formar o Product Backlog: os requisitos em forma de histórias do usuário serão elencados conforme o escopo proposto pelo sistema e com base no No-Excel, pois algumas funcionalidades serão iguais;
- Idealização da arquitetura do sistema e prototipagem do sistema**:** projetar o sistema em sua versão *web*, considerando diagramas e as telas do sistema

durante as Sprints;

- Desenvolvimento do sistema e testes do sistema: as funcionalidades serão desmembradas em Sprints, sendo que durante o desenvolvimento das histórias de uma Sprint, também serão executados os testes das funcionalidades.
- Validação dos requisitos implementados: todos os requisitos implementados serão validados com os clientes para verificar o atingimento do objetivo a cada final de uma Sprint.

Na figura 1 é possível observar o funcionamento do ciclo do Scrum:

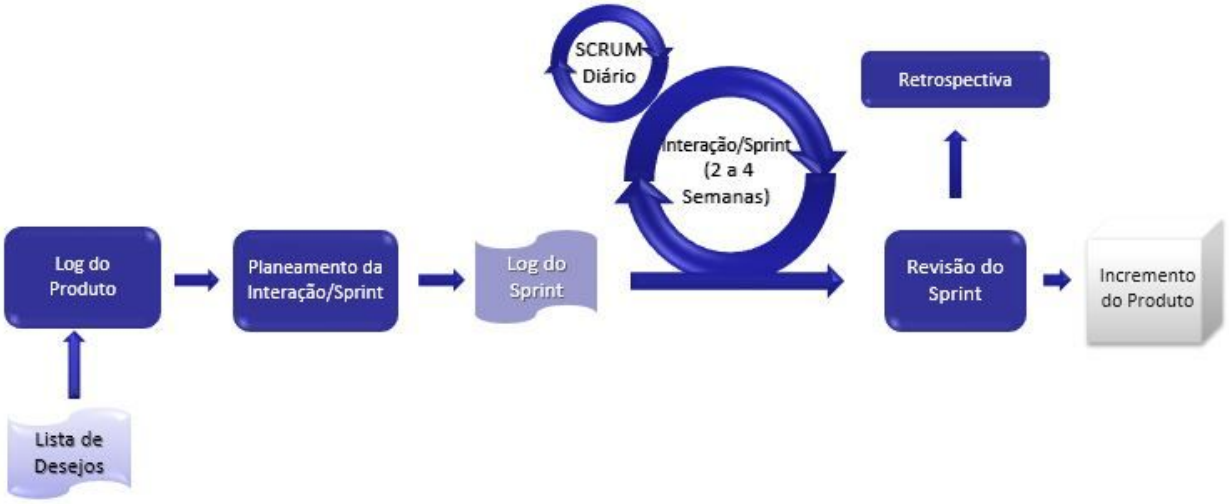

Figura 4 - Ciclo do Scrum

5 Análise e Projeto do Sistema

Neste capítulo serão apresentados os artefatos resultantes da análise e projeto do sistema. Mesmo que esteja se adotando uma metodologia ágil para o desenvolvimento deste projeto, o autor considera interessante projetar pelo menos o mínimo possível a priori a fim de se ter uma visão geral e ampla do escopo do sistema a ser desenvolvido. Esta visão abstrata do projeto é importante tanto para a equipe de desenvolvimento quanto para o cliente visualizar com uma certa antecedência as partes principais do

sistema. Neste sentido, o detalhamento de cada funcionalidade será trabalhado à medida em que as Sprints vão sendo desenvolvidas.

Este capítulo está organizado nas seguintes seções: na seção 5.1 será apresentado o Product Backlog resultado da fase tradicional de análise do sistema e na seção 5.2 serão apresentados os artefatos de projeto do sistema das funcionalidades mais importantes. . Foram desenvolvidos os protótipos das telas para o sistema e além disso foi construído o modelo para o banco de dados.

5.1 Análise do Sistema

A análise do sistema se iniciou pela definição do escopo do sistema. Por ser um trabalho acadêmico, a preocupação foi definir o escopo para que fosse possível implementar as referidas funcionalidades no tempo disponível para tal. Neste sentido, as principais funcionalidades do RU Digital consiste em permitir que que clientes consigam inserir créditos de forma autônoma e possam acompanhar seu extratos de qualquer período online. Além disso possibilitará que o administrador ou proprietário do restaurante consiga cadastrar gerentes para colaborarem nas atividades administrativas do sistema, principalmente para que tenham permissão para fazer os débitos nas contas dos usuários de acordo com as refeições consumidas.Vale salientar que tanto o administrador quanto os gerentes poderão lançar movimentações dos clientes, mas somente o administrador poderá cadastrar os produtos e seus preços.

As histórias do usuário foram extraídos com base no projeto No-Excel e conforme o acompanhamento do processo de funcionamento do restaurante universitários da UTFPR de Guarapuava. Para melhor clareza, as histórias dos usuários foram separadas conforme os papéis que as exercem, ou seja, histórias do cliente do restaurante, dos administradores ou gerentes e do visitante do restaurante. Neste domínio, um visitante é considerado como um cliente externos à universidade, ou seja, que não tem registro acadêmico.

Neste sentido, as histórias dos clientes estão apresentadas na Tabela 1. As histórias do administrador e gerente na Tabela 2 e por fim, as histórias dos visitantes

estão expressas na Tabela 3.

Tabela 1 Histórias do Cliente

1 - Eu como cliente desejo confirmar minha condição de bolsista através do envio de um comprovante.	**2 -** Eu como cliente desejo efetuar o meu cadastro no sistema.	**3 -** Eu como cliente desejo editar meu perfil e adicionar uma foto ou avatar no mesmo.
4 - Como cliente desejo recuperar minha senha em caso de esquecimento.	**5 -** Eu como cliente bolsista desejo ter meu desconto para consumos.	**6 -** Eu como cliente desejo acompanhar meu extrato da conta conforme período a definir.
7 - Eu como cliente desejo acompanhar o resumo da minha conta com informações do saldo.	**8 -** Eu como cliente desejo adicionar saldo a minha conta de forma online.	**9 -** Eu como cliente desejo saber o cardápio do dia bem como a localização do restaurante.

Tabela 2 Histórias do Administrador e do Gerente

1 - Eu como administrador desejo cadastrar o preço de acordo com um determinado grupo de clientes.	**2 -** Eu como administrador desejo excluir um colaborador (gerente).	**3 -** Eu como administrador desejo obter relatório financeiro sobre os valores entrados no sistema em um determinado período.
4- Eu como proprietário ou gerente do sistema desejo lançar consumos dos clientes.	**5 -** Eu como proprietário do restaurante desejo cadastrar gerentes no sistema.	**6 -** Eu como proprietário do restaurante desejo cadastrar e editar produtos disponíveis para consumo no restaurante.
7 - Eu como proprietário ou gerente desejo creditar saldo na conta de clientes que efetuam pagamento no balcão.	**8 -** Eu como proprietário ou gerente desejo editar o cardápio disponível no sistema.	

Tabela 3 História dos Visitantes

1 - Como visitante desejo conhecer sobre o funcionamento do sistema.	**2 -** Eu como visitante desejo entrar em contato com o responsável do restaurante.	

A partir das histórias levantadas, foi definido o primeiro Sprint Backlog. As demais Sprints Backlog serão definidas a posteriori, principalmente nas reuniões de entrega das funcionalidades implementadas nas sprints. As histórias a fazerem parte da primeira Sprint são:

- **2-** Eu como cliente desejo efetuar o meu cadastro no sistema.
- **7-** Eu como cliente desejo acompanhar o resumo da minha conta com informações do saldo.

5.2 PROJETO DO SISTEMA

Nesta seção será abordado como ocorrerá a comunicação da aplicação com o servidor e com o banco de dados. Além de identificar as telas do sistema bem como o modelo de banco de dados a ser seguido.

5.2.1 Projeto da Arquitetura do Sistema

Com o objetivo de facilitar o entendimento da comunicação entre o modelo, a visão, o controlador, o banco de dados e o servidor foi criado um desenho sobre a arquitetura geral do sistema, como pode ser visto na Figura 4.

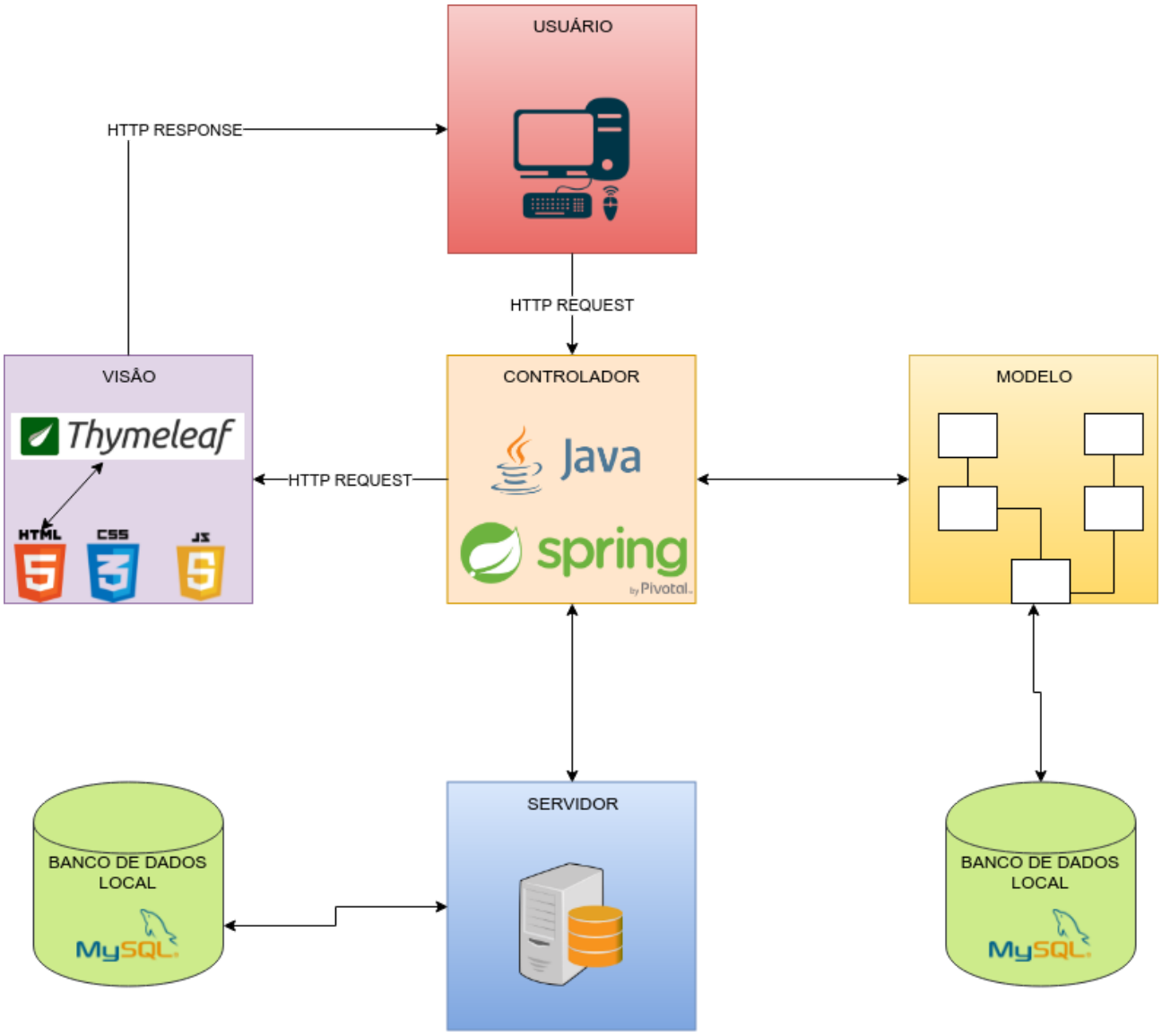

Figura 4: Projeto da Arquitetura do Sistema

Fonte: O autor (2018)

5.2.2 Projeto das Telas do Sistema

Esta apresenta o projeto das principais telas do sistema RU Digital. Mesmo que se esteja usando uma metodologia ágil, o projeto das telas ajuda ao cliente ter uma noção de como ficará o sistema e principalmente para organizar as transições entre telas e as entradas e saídas de dados. Na Figura 5 é apresentada a tela inicial de visitante do site.

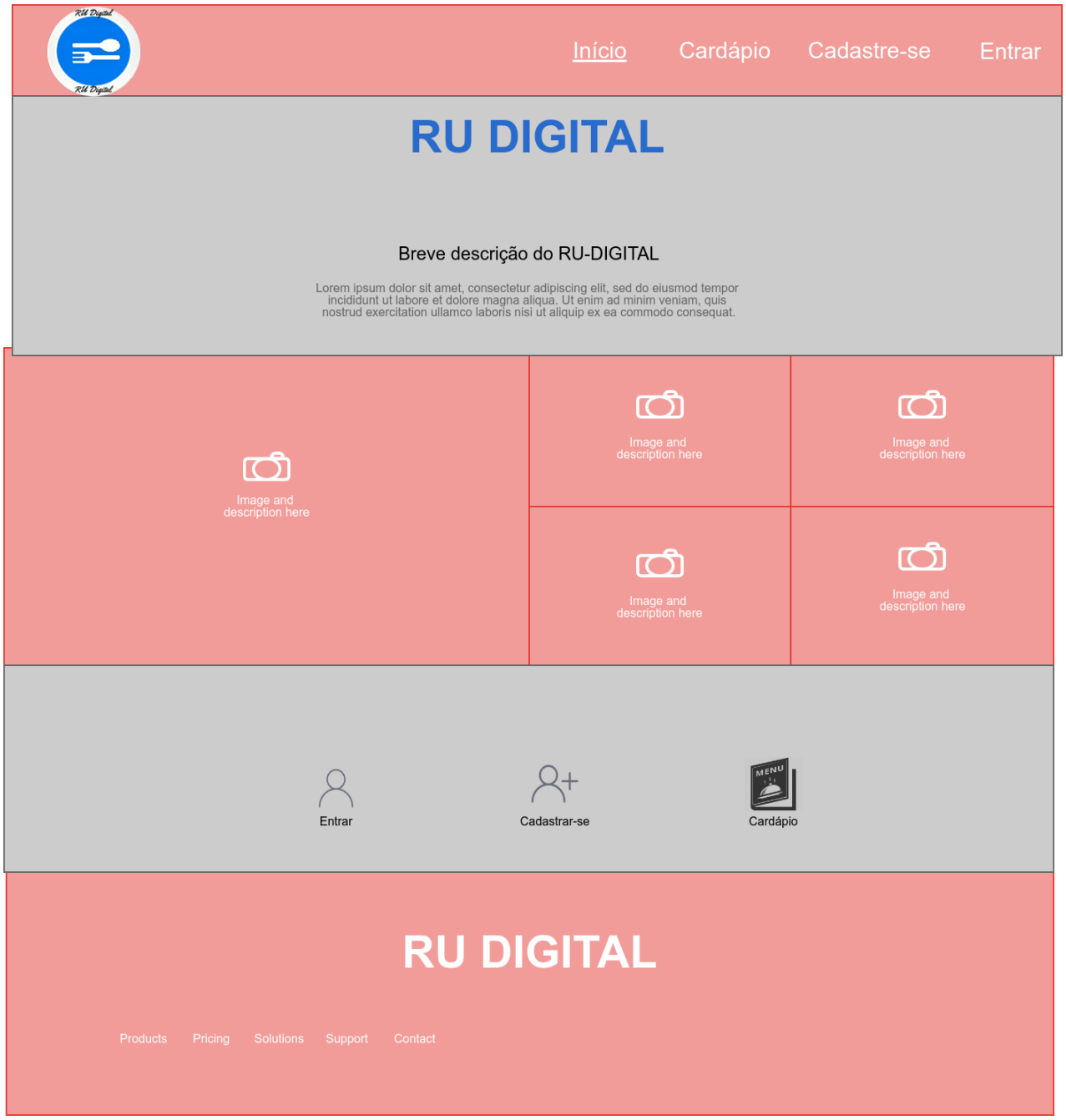

Figura 5: Ru-Digital: Tela inicial do Sistema

Fonte:O autor (2018)

Na Figura 6, Figura 7, Figura 8 e Figura 9 é possível verificar os projetos das principais telas relacionadas aos clientes dos restaurantes. Mais precisamente na Figura 6 é possível verificar a tela de cadastro de clientes, onde o mesmo deve fornecer suas informações básicas como nome, email, vínculo com a instituição, registro acadêmico e

senha.

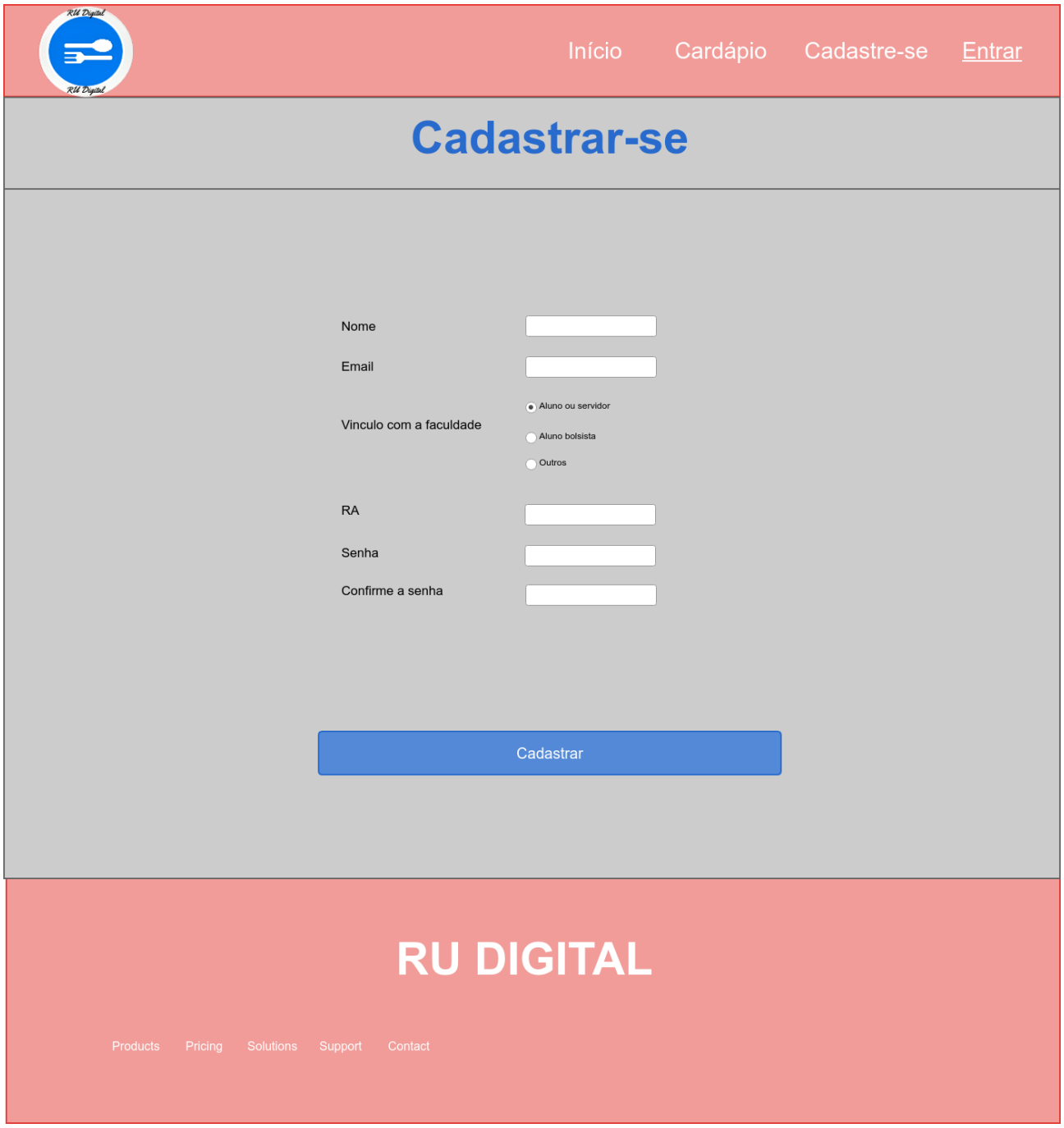

Figura 6: RU Digital: Tela de Cadastro de Clientes

Fonte: O autor (2018)

Na Figura 7 é ilustrado a tela inicial do cliente ao sistema (tela da conta do usuário) . Esta tela contém informações básicas relacionadas ao acesso no sistema, o avatar ou

foto definida para perfil e o saldo disponível em conta.

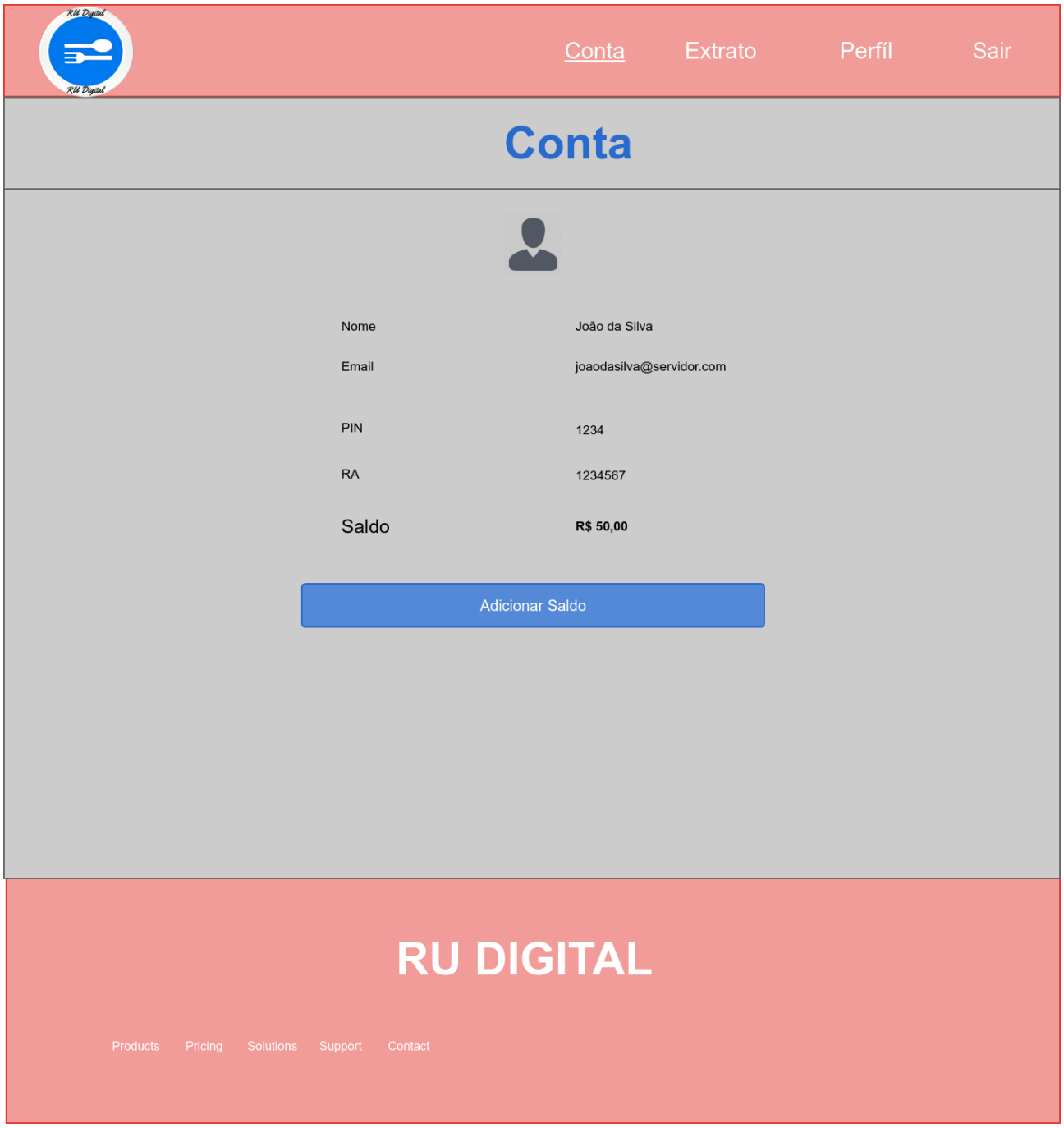

Figura 7: RU DIgital: Tela de Conta do Usuário

Fonte: O autor(2018)

Na Figura 8 está disposta a tela de edição de conta dos usuários, caso o cliente necessite alterar seus dados pessoas e de acesso.

Editar Conta do Cliente

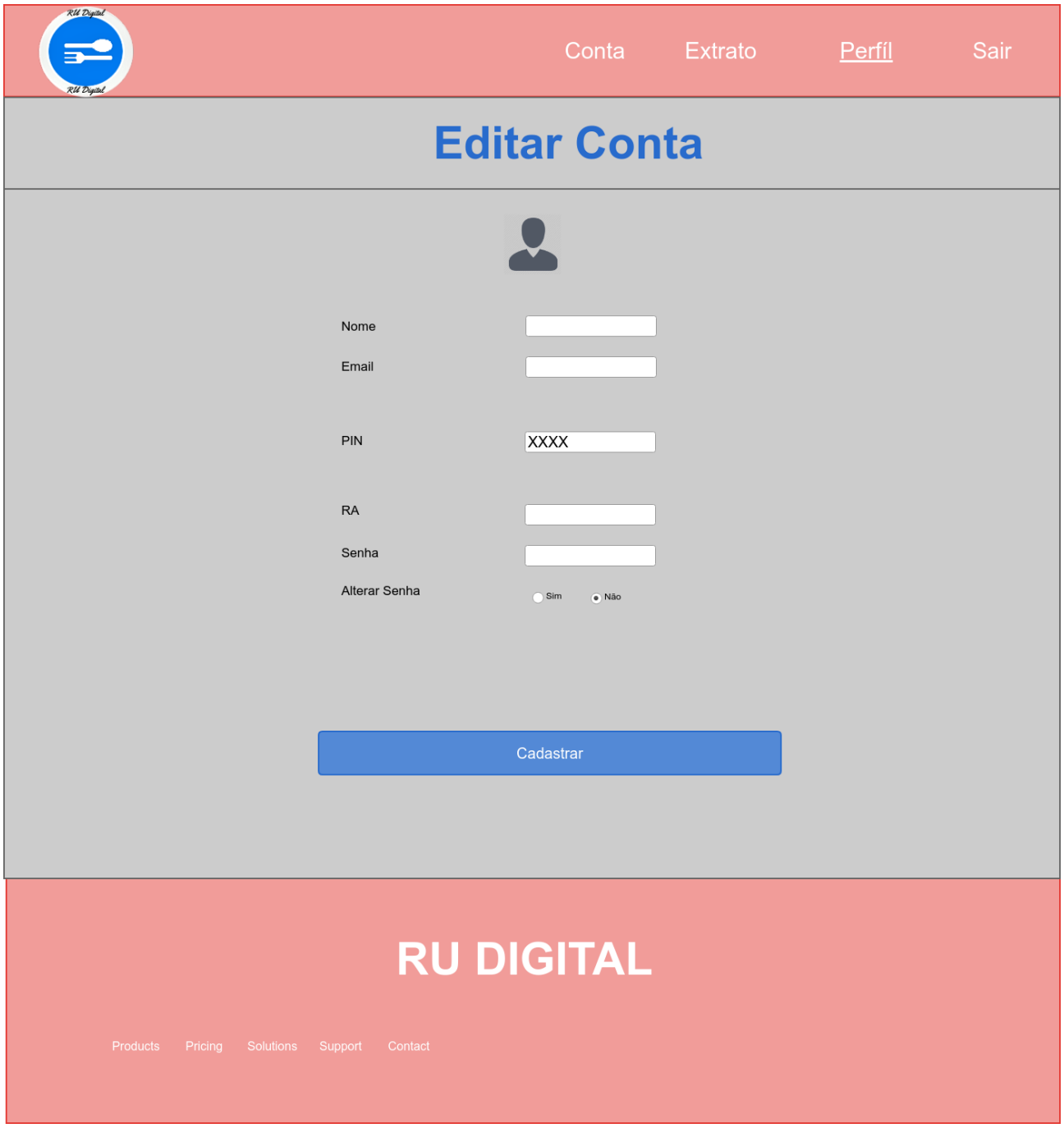

Figura 8: RU Digital: Tela de Edição de Conta do Cliente

Fonte: O autor (2018)

É possível verificar na Figura 9 o protótipo da tela de extrato do cliente. Para gerar um extrato, basta o cliente fornecer o período desejado e as informações são

apresentadas em uma tabela logo abaixo, a qual contém informações do operador da transação e o valor total. Caso o cliente deseje um extrato mais detalhado, com os produtos contidos em determinada transação, ele pode verificar no link com o título “expandir” de cada linha da tabela.

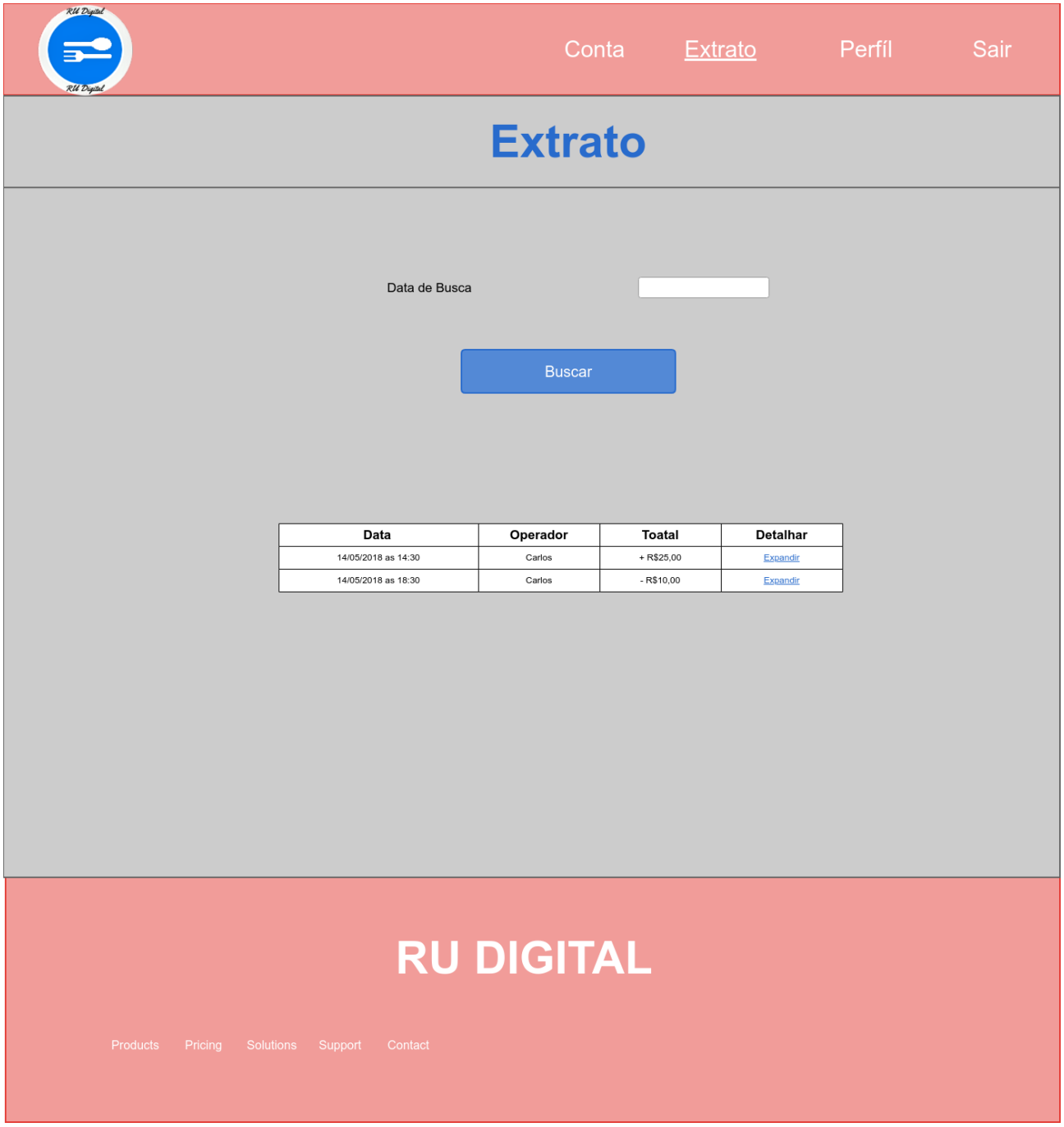

Figura 9: RU Digital Tela de Extrato de Consumo do Cliente

Fonte: O autor (2018)

Para que os perfis de cliente, gerente e administrador possam acessar o sistema, será necessário uma página de acesso. Para tanto, foi prototipado a tela de login como pode ser observado na Figura 10.

Login no Sistema

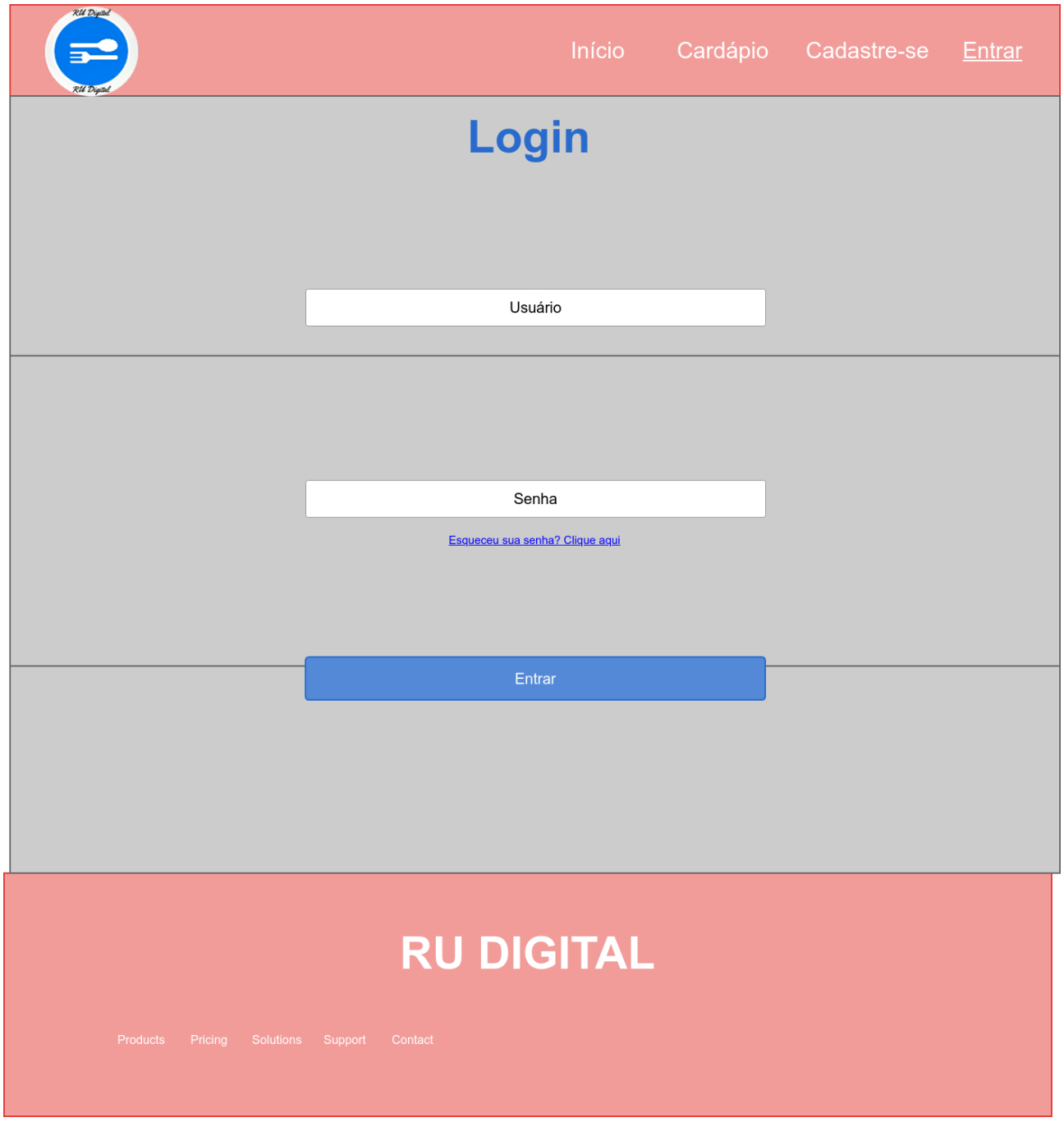

Figura 10: RU Digital Tela de Login dos Usuários

Fonte: O autor (2018)

As páginas do administrador e dos gerentes também foram projetadas. Na Figura 11 é possível observar a tela para cadastro de gerentes no sistema, este cadastro é feito pelo administrador do sistema.

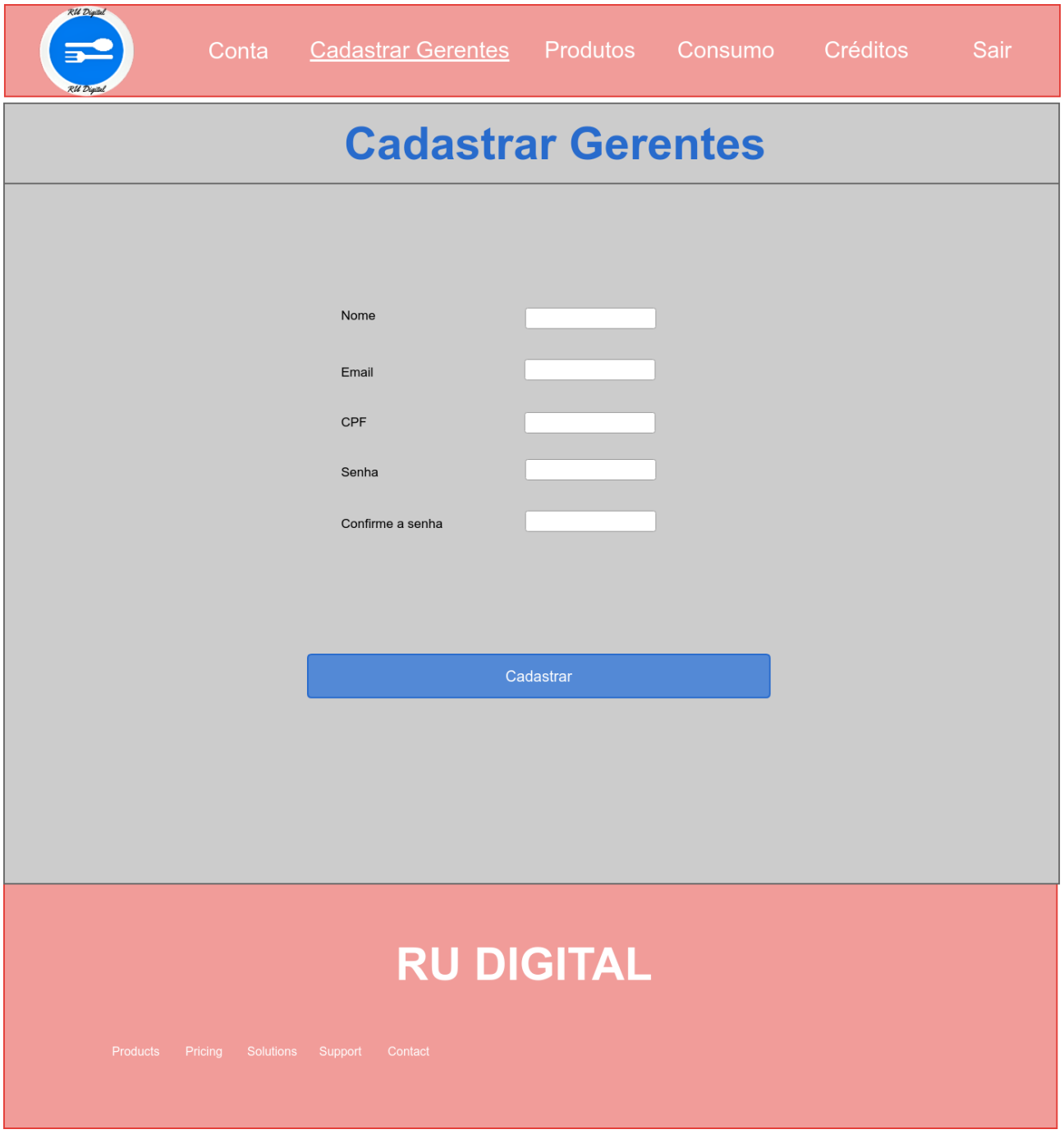

Figura 11: RU Digital Tela de Extrato de Cadastro do Gerente

Fonte: O autor (2018)

Para inserir créditos no sistema ao cliente, o administrador ou gerente do sistema devem primeiramente buscar pelo cliente através de seu RA ou CPF. Após buscar, o cliente é exibido logo abaixo e torna-se possível adicionar uma quantia de saldo para o

cliente como mostrado na Figura 12.

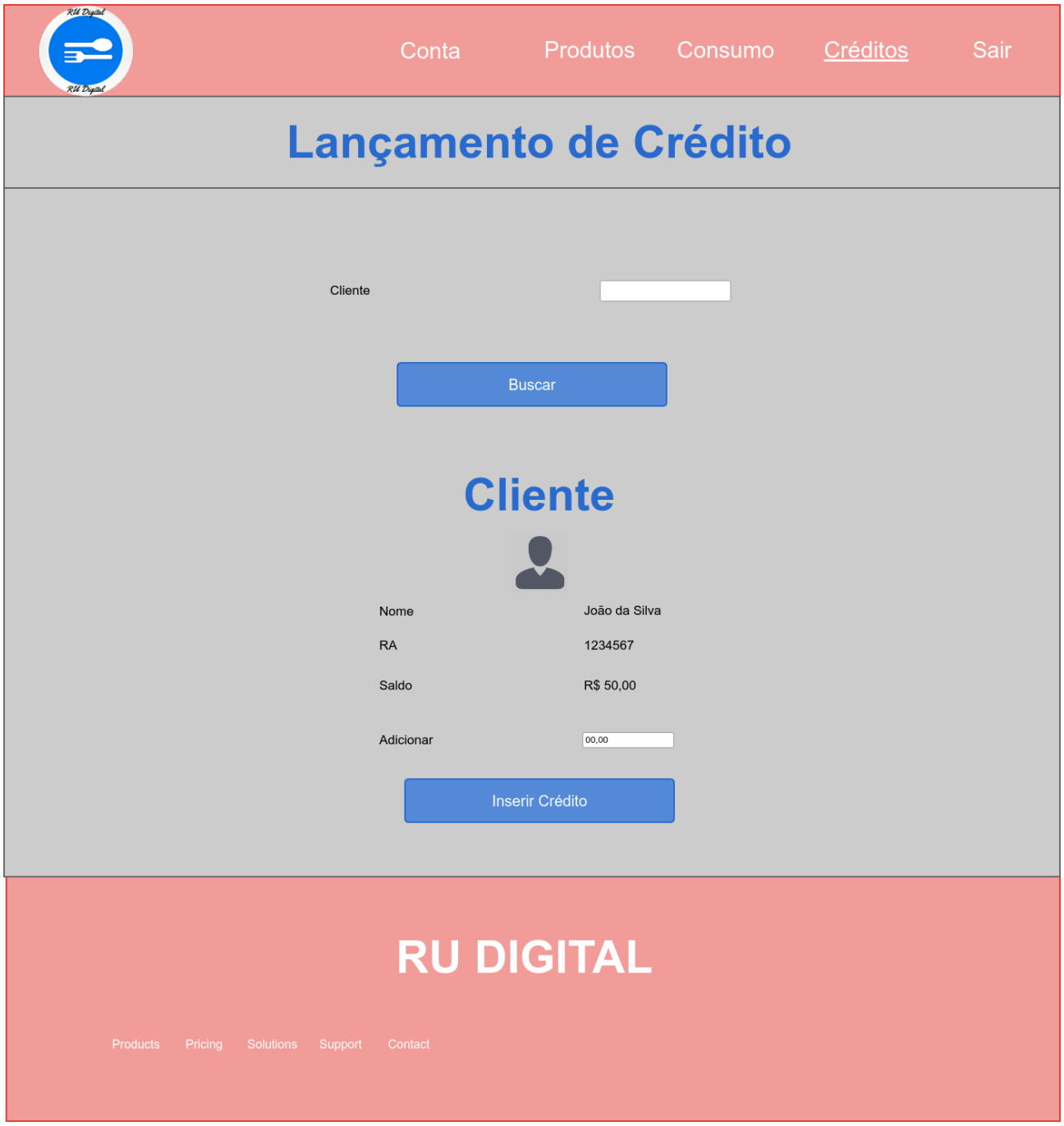

Figura 12: RU Digital Tela de Extrato de Inserção de Créditos para Clientes

Fonte: O autor (2018)

Na Figura 13 é exibida a tela de lançamento de consumos dos clientes. Nesta, o administrador deve buscar pelo cliente e em seguida pode adicionar os produtos

consumidos.Na sequência, o sistema verifica a disponibilidade de saldo do cliente e caso o mesmo não possua a quantidade suficiente, uma mensagem é exibida. Caso contrário, o valor é debitado da conta do cliente.

Figura 13: RU Digital Tela de Lançamento de Consumos

Fonte: O autor (2018)

Para cadastrar os produtos e seus respectivos preços foi projetada uma tela onde primeiramente é cadastrado um produto e na sequência pode-se cadastrar os preços de cada produto conforme o grupo de usuários. Esta tela está ilustrada na Figura 14.

Figura 14: RU Digital Tela de Cadastro de Produtos e Preços

Fonte: O autor (2018)

5.2.3 Projeto do Banco de Dados

No projeto do banco de dados, foi considerado a tabela de usuário (*user*) que está relacionado a muitas permissões (*roles*), ou seja, um usuário pode ter muitas permissões, e uma permissão pode ter muitos usuários. Por tratar-se de uma relação n para n, foi definido uma tabela intermediária chamada *user_roles*.

Caso um usuário esqueça a senha, é necessário recuperar e salvar um token na tabela *forgot_passwords* ou seja um usuário está relacionado a muitos tokens.

Um ou mais produtos (*products*) podem ter diferentes preços para cada grupo de usuário. Para isso, se fez necessário criar uma tabela intermediária chamada *product_prices*. Um usuário pode fazer uma ou várias transações (1 para n). Porém, muitas transações podem ser de crédito (*transaction_credit*) ou de débito (*transaction_debit*). A transação de débito precisa do preço do produto, por isso está relacionada com a tabela *product_prices*.

Como o sistema conta com um cardápio semanal dinâmico, foi necessário criar uma tabela para o mesmo no modelo. Esta tabela se chama *menu*. Na Figura 15 é ilustrado o modelo do banco de dados.

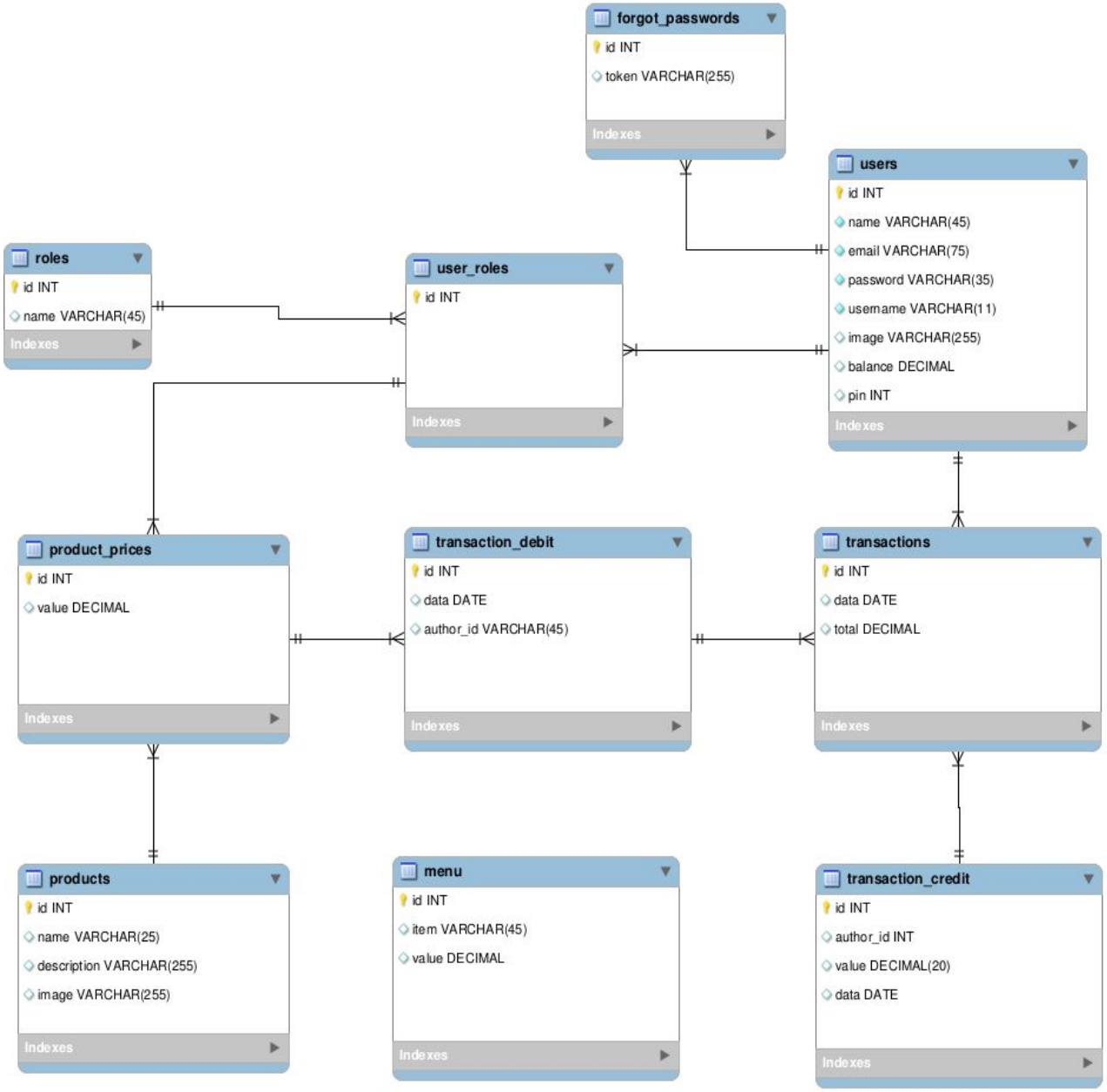

Figura 14: RU Digital Modelo do Banco de dados

Fonte: O autor (2018)

5.3 Conclusão

Nesta seção foram definidas os requisitos do sistema em forma de histórias do usuário conforme a metodologia Scrum e também foram definidas as principais telas que o sistema deverá possuir. Para o armazenamento dos dados, foi desenvolvido o modelo que será seguido do sistema. Mesmo que seja usado as práticas de ORM, tentará-se-á manter o esquema apresentado como resultado das tabelas e relacionamentos gerados

automaticamente pelo ORM. Vale ressaltar que tanto as histórias quanto os protótipos das telas e o modelo do banco poderão sofrer alterações no decorrer do desenvolvimento.

6 CONSIDERAÇÕES FINAIS

Este trabalho apresentou a proposta e projeto de um sistema de gerenciamento de saldo de clientes em restaurantes universitários, denominado RU Digital. O RU Digital possibilitará aos gerentes dos restaurantes acompanhar e gerenciar os saldos dos seus clientes, bem como ter relatórios administrativos sobre as transações gerais ocorridas viá saldo virtual. Também, o RU Digital possibilitará aos clientes acompanhar seus extratos e manipular seus saldos com confiabilidade e segurança. O sistema trará grandes vantagens à comunidade acadêmica, contribuindo principalmente para a redução do tempo de espera nas filas por agilizar o processo de pagamento de refeições.

7 REFERÊNCIAS

CIELO. **Escolha a Solução ideal para você realizar vendas e pagamentos com o cartão de crédito e débito.** Disponível em: <https://www.cielo.com.br/aceite-cartao/> Acesso em 04 de abril de 2018.

DOS SANTOS, José Airton Azevedo; ALVES, Roberta; POSSAN, Edna; SHIMIDT, Carla Adriana Pizarro.**Simulação da Dinâmica Operacional de um Pequeno Restaurante Universitário: Um Estudo de Caso.** Disponível em: <https://revistas.unicentro.br/index.php/RECEN/article/viewFile/1933/2139>. Acesso em: 04 abril de 2018.

FILHO, Luiz Felipe Basile Ribeiro. **No-Excel: Um sistema para gerenciamento e controle de saldo pré-carregado para restaurantes universitários.** Disponível em: <https://drive.google.com/file/d/1F-gK5ujRDFeONevxUZg89cKvfvWPolmz/view?usp=sharing>. Acesso em: 04 de abril de 2018.

PAGSEGURO. **Moderninha Pro**. Disponível em: <https://pagseguro.uol.com.br/sobre/taxas-e-tarifas?_ga=2.166843258.57538406.1523660747-1833154529.1519871261>. Acesso em 04 de abril de 2018.

QUESTIONÁRIO. **Formulário RU/Lanchonete da UTFPR**. Disponível em: <https://docs.google.com/forms/d/1xbWxjEggNyBF8pXYgNQID44gkCVnrnSJHpJsG3hb6r

o/edit#responses>. Acesso em 04 de abril de 2018

RUCARD, Usp. **Aquisição de créditos Rucard por meio de boleto bancário.** Disponível em: <http://cetilq.sti.usp.br/wp-content/uploads/sites/49/2017/02/Boleto-Marte-Web-Rucard.pdf >. Acesso em: 04 de abril de 2018.

RUCARD, Usp. **Rucard**. Disponível em: <https://uspdigital.usp.br/rucard/>. Acesso em: 04 de abril de 2018.

SRU, Unesp. **Perguntas e Respostas Frequentes (FAQ)**. Disponível em: <https://sru.ibilce.unesp.br>. Acesso em: 04 de abril de 2018.

SRU, Unesp. **Cardápio** . Disponível em: <https://sru.ibilce.unesp.br>. Acesso em: 04 de abril de 2018.

UFPR. **Breve Histórico**. Disponível em: <http://www.pra.ufpr.br/portal/ru/historico/>. Acesso em: 04 de abril de 2018.

WHITE, Oliver .**Top 4 Java Web Frameworks Revealed Real Life Usage Data Of Spring MVC Vaadin GWT And JSF**. Disponível em: <https://zeroturnaround.com/rebellabs/top-4-java-web-frameworks-revealed-real-life-usage-data-of-spring-mvc-vaadin-gwt-and-jsf/>. Acesso em 04 de abril de 2018.

DUCKETT, Jon. **HTML e CSS Projete e Construa Websites.** 1º Edição. Altabooks, 2015.

EIS, DIego. **O básico: O que é HTML?**. Disponível em <https://tableless.com.br/o-que-html-basico/>. Acesso em: 15 de maio de 2018.

SHIMITT, Christopher. **CSS Cookbok.** 1º Edição. Novatec, 2010.

MATERIALIZECSS. **About.** Disponível em <https://materializecss.com/about.html>. Acesso em: 15 de maio de 2018.

MOURA, Juliano. **Materialize: O que é e para que serve este framework?.** Disponível em <https://www.ciawebsites.com.br/sites/o-que-e-materialize/>. Acesso em: 15 de maio de 2018.

DUCKETT, Jon. **Javascript e Jquery, Desenvolvimento de Interfaces Web Interativas.** 1º Edição. Altabooks, 2016.

NODEJS. **About Node.js**. Disponível em <https://nodejs.org/en/about/>. Acesso em: 15 de maio de 2018.

AFONSO, Alexandre. **Produtividade no Desenvolvimento de Aplicações Web Com Spring Boot.** 3º Edição. Algaworks, 2017.

STS. **Tools.** Disponível em <https://spring.io/tools>. Acesso em: 15 de maio de 2018.

MAVEN. **Introduction.** Disponível em <https://maven.apache.org/what-is-maven.html>. Acesso em: 15 de maio de 2018.

PISA, Pedro. **O que é e como usar o Mysql? | Artigos | TechTudo.** Disponível em <http://www.techtudo.com.br/artigos/noticia/2012/04/o-que-e-e-como-usar-o-mysql.html>. Acesso em 15 de maio de 2018.

TOMCAT. Apache. **Apache Tomcat.** Disponível em <http://tomcat.apache.org/>. Acesso em 15 de maio de 2018.

SWARTZ, Baron; ZAITZEV, Peter; TKACHENKO, Valdin. **Alto Desempenho em Mysql.** 2º Edição. Altabook, 2009.

BCRYPT-OFFICIAL. **Nuget Gallery.** Disponível em <https://www.nuget.org/packages/BCrypt-Official/>. Acesso em 15 de maio de 2018.

BCRYPT. **Bcrypt.** Disponível em <https://docs.spring.io/spring-security/site/docs/5.0.0.M4/api/org/springframework/security/crypto/bcrypt/BCrypt.html>. Acesso em 15 de maio de 2018.

OWSAP. **Top 10 Application Security Risk - 2017.** Disponível em <https://www.owasp.org/index.php/Top_10-2017_Top_10>. Acesso em 15 de maio de 2018.

LETSENCRYPT. Let's Encrypt - Free SSL/TLS Certificates. Disponível em <https://letsencrypt.org/> acesso em 15 de maio.